ICPC Team Reference Document

HCMUS-mailsinhvien

Tran Thien Phuc
Ho Phong Linh
Huynh Ngoc Nhan



University of Science - VNUHCM

1 Linux Terminal

• 1.1 Kill A Loop

```
Ctrl + C in VS Code
```

• 1.2 Run Script

```
#!/bin/bash
NAME="${1:-code}"
g++ -O2 -std=c++17 "${NAME}.cpp" -o "${NAME}" && ./"${NAME}"

# Run once the first time:
# chmod +x run.sh

# For other turns:
# ./run.sh name
# Ex: ./run.sh code for running code.cpp
```

2 Debug

• 2.1 Linux C++

```
#include <bits/stdc++.h>
using namespace std;

#define file "TASK"
const int NTEST = 100;

mt19937_64 rng(chrono::high_resolution_clock::now().
    time_since_epoch().count());
long long Rand(long long l, long long r) {
  return uniform_int_distribution<long long>(l, r)(rng);
}

void makeTest() {
  ofstream cout(file".inp");

}

int main() {
  ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
  srand(time(NULL));

  system("g++ -O2 " file ".cpp -o " file);
  system("g++ -O2 " file "_brute.cpp -o " file "_brute");

  for (int idT = 1; idT <= NTEST; idT++) {
    makeTest();
    system("./" file " < " file ".inp > " file ".out");
    system("./" file "_brute < " file ".inp > " file ".ans")
    ;

    cerr << "Test " << idT << ": ";

    if (system("diff -q -Z " file ".out " file ".ans > /dev/
    null") != 0) {
      cerr << "WA\n";
      cerr << "\nCheck " file ".inp, " file ".out, and "
    file ".ans\n";
      return 0;
    }
    cerr << "AC\n";
  }

  cerr << "\nPassed all " << NTEST << " tests!\n";
  return 0;
}
```

• 2.2 Linux Shell

```
set -e
g++ code.cpp -o code
g++ gen.cpp -o ge
g++ brute.cpp -o brute
for((i = 1; ; ++i)); do
  ./gen $i > input_file
  ./code < input_file > myAnswer
  ./brute < input_file > correctAnswer
  diff -Z myAnswer correctAnswer > /dev/null || break
  echo "Passed test: " $i
done
echo "WA on the following test:"
cat input_file
echo "Your answer is:"
cat myAnswer
echo "Correct answer is:"
cat correctAnswer

# Run once the first time:
# chmod +x debug.sh

# For other turns:
# ./debug.sh
```

• 2.3 Windows C++

```
#include <bits/stdc++.h>
using namespace std;
#define file "TASK"
const int NTEST = 100;

mt19937_64 rng(chrono::high_resolution_clock::now().
    time_since_epoch().count());
long long Rand(long long l, long long r) {
  return uniform_int_distribution <int> (l, r) (rng);
}

void makeTest() {
  ofstream cout(file".inp");

}

signed main() {
  ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
  srand(time(NULL));

  for (int idT = 1; idT <= NTEST; idT++) {
    makeTest();
    system(file".exe");
    system(file"_brute.exe");

    cerr << "Test " << idT << ": ";
    if (system("fc "file".out "file".ans")) {
      cerr << "WA\n";
      return 0;
    }
    cerr << "AC\n";
  }
  return 0;
}
```

3 Data Structure

• 3.1 Fast Set

```
struct FastSet {
  static constexpr int BASE = 6;
  static constexpr int RANGE = 1 << BASE;
  static constexpr int FILTER = RANGE - 1;
  int N, log;
  vector<vector<u64>> seg;
  FastSet(int _N) { build(_N); }
  void build(int _N) {
    seg.clear();
    N = _N;
    while(true) {
      seg.emplace_back((_N + FILTER) >> BASE, 0);
      if(seg.back().size() <= 1) break;
      _N = seg.back().size();
    }
  }
  void insert(int k) {
    for(int i = 0; i < seg.size(); ++i, k >>= BASE) {
      if(seg[i][k >> BASE] >> (k & FILTER) & 1) break;
      seg[i][k >> BASE] ^= u64(1) << (k & FILTER);
    }
  }
  void erase(int k) {
    u64 x = 0;
    for(int i = 0; i < seg.size(); ++i, k >>= BASE) {
      seg[i][k >> BASE] &= ~(u64(1) << (k & FILTER));
      seg[i][k >> BASE] |= x << (k & FILTER);
      x = bool(seg[i][k >> BASE]);
      if(x) break;
    }
  }
  bool operator[](int k) {
    assert(0 <= k && k < N);
    return seg[0][k >> BASE] >> (k & FILTER) & 1;
  }
  int next(int k) {
    k = max(0, k);
    for(int i = 0; i < seg.size(); ++i) {
      if((k >> BASE) >= seg[i].size()) break;
      u64 x = seg[i][k >> BASE] >> (k & FILTER);
      if(!x) { ++(k >>= BASE); continue; }
      k += __builtin_ctzll(x);
      for(int j = i - 1; j >= 0; --j) k = (k << BASE) |
    __builtin_ctzll(seg[j][k]);
      return k;
    }
    return N;
  }
  int prev(int k) {
    k = min(k, N - 1);
    for(int i = 0; i < seg.size(); ++i) {
      if(k < 0) break;
      u64 x = seg[i][k >> BASE] << (63 - (k & FILTER));
      if(!x) { --(k >>= BASE); continue; }
      k -= __builtin_clzll(x);
      for(int j = i - 1; j >= 0; --j) k = (k << BASE) | (63
    - __builtin_clzll(seg[j][k]));
      return k;
    }
    return -1;
  }
  template<typename F>
  void enumerate(int L, int R, F f) {
    for(int i = next(L); i < R; i = next(i + 1)) f(i);
  }
};
```

• 3.2 Heavy Light Decomposition

```
struct Heavy_light_decomposition {
  int head[N], id[N];
  int idCnt = 0;

  void dfs(int u, int pre, int top) {
    id[u] = ++idCnt;
    head[u] = top;

    int heavySz = 0, heavy = -1;
    for (int v: adj[u]) if (v != pre) if (maxi(heavySz, sz[v
    ])) heavy = v;

    if (heavy == -1) return;
    dfs(heavy, u, top);
```

```
    for (int v: adj[u]) if (v != pre and v != heavy) dfs(v,
    u, v);
  }

  void init() {
    dfs(1, 1, 1);
    for (int u = 1; u <= n; u++) valHld[id[u]] = a[u];
    seg.init(1, 1, n);
  }

  void update(int pos, int val) {
    seg.update(1, 1, n, id[pos], val);
  }

  int get(int u, int v) {
    int res = -inf;

    while (head[u] != head[v]) {
      if (id[u] > id[v]) swap(u, v);
      maxi(res, seg.get(1, 1, n, id[head[v]], id[v]));
      v = par[head[v]];
    }

    if (id[u] > id[v]) swap(u, v);
    maxi(res, seg.get(1, 1, n, id[u], id[v]));
    return res;
  }
} hld;
```

• 3.3 Matrix

```
struct Matrix {
  int n, m;
  vector <vector <ll>> val;

  Matrix(int _n = 0, int _m = 0) : n(_n), m(_m) {
    val.assign(n + 5, vector <ll> (m + 5, 0));
  }

  Matrix(vector <vector <ll>> _val) : val(_val) {
    n = Sz(val);
    m = Sz(val[0]);
  }

  vector <ll> & operator [] (const int &id) {
    return val[id];
  }

  Matrix operator * (Matrix ot) {
    Matrix res(n, ot.m);

    for (int i = 0; i < n; i++) {
      for (int j = 0; j < ot.m; j++) {
        for (int k = 0; k < m; k++) {
          res[i][j] += val[i][k] * ot[k][j];
          if (res[i][j] >= modSqr) res[i][j] -= modSqr;
        }
      }
    }

    for (int i = 0; i < n; i++) {
      for (int j = 0; j < ot.m; j++) {
        res[i][j] %= mod;
      }
    }

    return res;
  }

  Matrix Pow(ll pw) {
    Matrix res(n, n);
    for (int i = 0; i < n; i++) res[i][i] = 1;

    Matrix A = (*this);

    while (pw) {
      if (pw & 1) res = res * A;
      A = A * A;
      pw >>= 1;
    }
    return res;
  }
};
```

• 3.4 Persistent Segment Tree

```
struct Persistent_segment_tree {
  int cntNode = 0;
  struct Node {
    ll sum;
    int left, right;

    Node() {
      sum = 0;
      left = 0;
      right = 0;
    }
  } d[N * 4 + N * LOG];
  vector <int> version;

  int build(int l, int r) {
    int id = ++cntNode;
    if (l == r) {
      d[id].sum = a[l];
      return id;
    }
    int mid = (l + r) >> 1;
    d[id].left = build(l, mid);
    d[id].right = build(mid + 1, r);
    d[id].sum = d[d[id].left].sum + d[d[id].right].sum;
    return id;
  }

  int update(int id, int l, int r, int pos, int val) {
    if (r < pos or l > pos) return id;
    int idx = ++cntNode;
    if (l == r and r == pos) {
      d[idx].sum = val;
      return idx;
    }
    int mid = (l + r) >> 1;
    if (pos <= mid) {
      d[idx].right = d[id].right;
      d[idx].left = update(d[id].left, l, mid, pos, val);
    }
    else {
      d[idx].left = d[id].left;
      d[idx].right = update(d[id].right, mid + 1, r, pos,
    val);
    }
    d[idx].sum = d[d[idx].left].sum + d[d[idx].right].sum;
    return idx;
  }

  ll get(int id, int l, int r, int x, int y) {
    if (r < x or l > y) return 0;
    if (x <= l and r <= y) return d[id].sum;
    int mid = (l + r) >> 1;
    return get(d[id].left, l, mid, x, y) + get(d[id].right,
    mid + 1, r, x, y);
  }

  void init() {
    version.push_back(build(1, n));
  }

  void update(int ver, int pos, int val) {
    version[ver] = update(version[ver], 1, n, pos, val);
  }

  void get(int ver, int l, int r) {
    cout << get(version[ver], 1, n, l, r) << "\n";
  }

  void copy(int ver) {
    int id = ++cntNode;
    version.push_back(id);
    d[id] = d[version[ver]];
  }
} seg;
```

4 DP

• 4.1 DP CHT

```
struct Convex_hull_trick {
  struct Line {
    ll a, b;

    double inter(const Line &ot) const {
      return 1.0 * (ot.b - b) / (a - ot.a);
    }

    ll eval(int x) {
      return a * x + b;
    }
  };

  vector <Line> hull;
  int ptr = 0;

  bool bad(const Line &pre, const Line &cur, const Line &nxt
    ) {
    return nxt.inter(pre) <= cur.inter(pre);
  }

  void reset() {
    hull.clear();
  }

  void add(ll a, ll b) {
    Line cur = {a, b};
    while (Sz(hull) >= 2 and bad(hull[Sz(hull) - 2], hull.
    back(), cur)) hull.pop_back();
    hull.pb(cur);
  }

  ll get(int x) {
    if (ptr >= Sz(hull)) ptr = Sz(hull) - 1;
    while (ptr < Sz(hull) - 1 and hull[ptr + 1].inter(hull[
    ptr]) <= x) ptr++;
    return hull[ptr].eval(x);
  }
} cht;
```

• 4.2 DP DNC

```
void dnc(int i, int l, int r, int optl, int optr) {
  if (l > r) return;
  int mid = (l + r) >> 1;
  pair <long long, int> best = {inf, l};
  for (int j = optl; j < min(mid, optr + 1); j++) {
    long long s = sum[mid] - sum[j];
    best = min(best, {s * s + f[i - 1][j], j});
  }
  f[i][mid] = best.first;
  int opt = best.second;
  dnc(i, l, mid - 1, optl, opt);
```

```
  dnc(i, mid + 1, r, opt, optr);
}
```

• 4.3 DP SOS

```
for (int i = 1; i <= n; i++) cnt[a[i]]++;

// Normal DP SOS Submask
for (int mask = 0; mask < (1 << 20); mask++) sosSub[mask] = cnt[mask];
for (int lg = 0; lg < 20; lg++) {
  for (int mask = 0; mask < (1 << 20); mask++) {
    if (getBit(mask, lg)) sosSub[mask] += sosSub[mask ^ (1 << lg)];
  }
}

// Normal DP SOS Supermask
for (int mask = 0; mask < (1 << 20); mask++) sosSup[mask] = cnt[mask];
for (int lg = 0; lg < 20; lg++) {
  for (int mask = 0; mask < (1 << 20); mask++) {
    if (!getBit(mask, lg)) sosSup[mask] += sosSup[mask ^ (1 << lg)];
  }
}

// Reverse DP SOS Submask
for (int mask = 0; mask < (1 << 20); mask++) revSub[mask] = sosSub[mask];
for (int lg = 0; lg < 20; lg++) {
  for (int mask = 0; mask < (1 << 20); mask++) {
    if (getBit(mask, lg)) revSub[mask] -= revSub[mask ^ (1 << lg)];
  }
}

// Reverse DP SOS Supermask
for (int mask = 0; mask < (1 << 20); mask++) revSup[mask] = sosSup[mask];
for (int lg = 0; lg < 20; lg++) {
  for (int mask = 0; mask < (1 << 20); mask++) {
    if (!getBit(mask, lg)) revSup[mask] -= revSup[mask ^ (1 << lg)];
  }
}
```

5 Big Int

• 5.1 Big Int

```
#include <vector>
#include <cstdlib>
#include <iostream>
#include <iomanip>
#include <string>
#include <ctime>
using namespace std;

// base and base_digits must be consistent
const int base = 1000000000;
const int base_digits = 9;

struct bigint {
  vector<int> a;
  int sign;

  bigint() :
    sign(1) {
  }

  bigint(long long v) {
    *this = v;
  }

  bigint(const string &s) {
    read(s);
  }

  void operator=(const bigint &v) {
    sign = v.sign;
    a = v.a;
  }

  void operator=(long long v) {
    sign = 1;
    if (v < 0)
      sign = -1, v = -v;
    for (; v > 0; v = v / base)
      a.push_back(v % base);
  }

  bigint operator+(const bigint &v) const {
    if (sign == v.sign) {
      bigint res = v;

      for (int i = 0, carry = 0; i < (int) max(a.size(), v.a.size()) || carry; ++i) {
        if (i == (int) res.a.size())
          res.a.push_back(0);
        res.a[i] += carry + (i < (int) a.size() ? a[i] : 0);
        carry = res.a[i] >= base;
        if (carry)
          res.a[i] -= base;
      }
      return res;
    }
    return *this - (-v);
  }

  bigint operator-(const bigint &v) const {
    if (sign == v.sign) {
      if (abs() >= v.abs()) {
        bigint res = *this;
        for (int i = 0, carry = 0; i < (int) v.a.size() || carry; ++i) {
          res.a[i] -= carry + (i < (int) v.a.size() ? v.a[i] : 0);
          carry = res.a[i] < 0;
          if (carry)
            res.a[i] += base;
        }
        res.trim();
        return res;
      }
      return -(v - *this);
    }
    return *this + (-v);
  }

  void operator*=(int v) {
    if (v < 0)
      sign = -sign, v = -v;
    for (int i = 0, carry = 0; i < (int) a.size() || carry; ++i) {
      if (i == (int) a.size())
        a.push_back(0);
      long long cur = a[i] * (long long) v + carry;
      carry = (int) (cur / base);
      a[i] = (int) (cur % base);
      //asm("divl %%ecx" : "=a"(carry), "=d"(a[i]) : "A"(cur), "c"(base));
    }
    trim();
  }

  bigint operator*(int v) const {
    bigint res = *this;
    res *= v;
    return res;
  }

  friend pair<bigint, bigint> divmod(const bigint &a1, const bigint &b1) {
    int norm = base / (b1.a.back() + 1);
    bigint a = a1.abs() * norm;
    bigint b = b1.abs() * norm;
    bigint q, r;
    q.a.resize(a.a.size());

    for (int i = a.a.size() - 1; i >= 0; i--) {
      r *= base;
      r += a.a[i];
      int s1 = r.a.size() <= b.a.size() ? 0 : r.a[b.a.size()];
      int s2 = r.a.size() <= b.a.size() - 1 ? 0 : r.a[b.a.size() - 1];
      int d = ((long long) base * s1 + s2) / b.a.back();
      r -= b * d;
      while (r < 0)
        r += b, --d;
      q.a[i] = d;
    }

    q.sign = a1.sign * b1.sign;
    r.sign = a1.sign;
    q.trim();
    r.trim();
    return make_pair(q, r / norm);
  }

  bigint operator/(const bigint &v) const {
    return divmod(*this, v).first;
  }

  bigint operator%(const bigint &v) const {
    return divmod(*this, v).second;
  }

  void operator/=(int v) {
    if (v < 0)
      sign = -sign, v = -v;
    for (int i = (int) a.size() - 1, rem = 0; i >= 0; --i) {
      long long cur = a[i] + rem * (long long) base;
      a[i] = (int) (cur / v);
      rem = (int) (cur % v);
    }
    trim();
  }

  bigint operator/(int v) const {
    bigint res = *this;
    res /= v;
    return res;
  }

  int operator%(int v) const {
    if (v < 0)
      v = -v;
    int m = 0;
    for (int i = a.size() - 1; i >= 0; --i)
      m = (a[i] + m * (long long) base) % v;
    return m * sign;
  }

  void operator+=(const bigint &v) {
    *this = *this + v;
```

```
  }
  void operator-=(const bigint &v) {
    *this = *this - v;
  }
  void operator*=(const bigint &v) {
    *this = *this * v;
  }
  void operator/=(const bigint &v) {
    *this = *this / v;
  }

  bool operator<(const bigint &v) const {
    if (sign != v.sign)
      return sign < v.sign;
    if (a.size() != v.a.size())
      return a.size() * sign < v.a.size() * v.sign;
    for (int i = a.size() - 1; i >= 0; i--)
      if (a[i] != v.a[i])
        return a[i] * sign < v.a[i] * sign;
    return false;
  }

  bool operator>(const bigint &v) const {
    return v < *this;
  }
  bool operator<=(const bigint &v) const {
    return !(v < *this);
  }
  bool operator>=(const bigint &v) const {
    return !(*this < v);
  }
  bool operator==(const bigint &v) const {
    return !(*this < v) && !(v < *this);
  }
  bool operator!=(const bigint &v) const {
    return *this < v || v < *this;
  }

  void trim() {
    while (!a.empty() && !a.back())
      a.pop_back();
    if (a.empty())
      sign = 1;
  }

  bool isZero() const {
    return a.empty() || (a.size() == 1 && !a[0]);
  }

  bigint operator-() const {
    bigint res = *this;
    res.sign = -sign;
    return res;
  }

  bigint abs() const {
    bigint res = *this;
    res.sign *= res.sign;
    return res;
  }

  long long longValue() const {
    long long res = 0;
    for (int i = a.size() - 1; i >= 0; i--)
      res = res * base + a[i];
    return res * sign;
  }

  friend bigint gcd(const bigint &a, const bigint &b) {
    return b.isZero() ? a : gcd(b, a % b);
  }
  friend bigint lcm(const bigint &a, const bigint &b) {
    return a / gcd(a, b) * b;
  }

  void read(const string &s) {
    sign = 1;
    a.clear();
    int pos = 0;
    while (pos < (int) s.size() && (s[pos] == '-' || s[pos]
    == '+')) {
      if (s[pos] == '-')
        sign = -sign;
      ++pos;
    }
    for (int i = s.size() - 1; i >= pos; i -= base_digits) {
      int x = 0;
      for (int j = max(pos, i - base_digits + 1); j <= i; j
    ++)
        x = x * 10 + s[j] - '0';
      a.push_back(x);
    }
    trim();
  }

  friend istream& operator>>(istream &stream, bigint &v) {
    string s;
    stream >> s;
    v.read(s);
    return stream;
  }

  friend ostream& operator<<(ostream &stream, const bigint &
    v) {
    if (v.sign == -1)
      stream << '-';
    stream << (v.a.empty() ? 0 : v.a.back());
    for (int i = (int) v.a.size() - 2; i >= 0; --i)
      stream << setw(base_digits) << setfill('0') << v.a[i];
    return stream;
  }

  static vector<int> convert_base(const vector<int> &a, int
    old_digits, int new_digits) {
    vector<long long> p(max(old_digits, new_digits) + 1);
    p[0] = 1;
    for (int i = 1; i < (int) p.size(); i++)
      p[i] = p[i - 1] * 10;
    vector<int> res;
    long long cur = 0;
    int cur_digits = 0;
    for (int i = 0; i < (int) a.size(); i++) {
      cur += a[i] * p[cur_digits];
      cur_digits += old_digits;
      while (cur_digits >= new_digits) {
        res.push_back(int(cur % p[new_digits]));
        cur /= p[new_digits];
        cur_digits -= new_digits;
      }
    }
    res.push_back((int) cur);
    while (!res.empty() && !res.back())
      res.pop_back();
    return res;
  }

  typedef vector<long long> vll;

  static vll karatsubaMultiply(const vll &a, const vll &b) {
    int n = a.size();
    vll res(n + n);
    if (n <= 32) {
      for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
          res[i + j] += a[i] * b[j];
      return res;
    }

    int k = n >> 1;
    vll a1(a.begin(), a.begin() + k);
    vll a2(a.begin() + k, a.end());
    vll b1(b.begin(), b.begin() + k);
    vll b2(b.begin() + k, b.end());

    vll a1b1 = karatsubaMultiply(a1, b1);
    vll a2b2 = karatsubaMultiply(a2, b2);

    for (int i = 0; i < k; i++)
      a2[i] += a1[i];
    for (int i = 0; i < k; i++)
      b2[i] += b1[i];

    vll r = karatsubaMultiply(a2, b2);
    for (int i = 0; i < (int) a1b1.size(); i++)
      r[i] -= a1b1[i];
    for (int i = 0; i < (int) a2b2.size(); i++)
      r[i] -= a2b2[i];

    for (int i = 0; i < (int) r.size(); i++)
      res[i + k] += r[i];
    for (int i = 0; i < (int) a1b1.size(); i++)
      res[i] += a1b1[i];
    for (int i = 0; i < (int) a2b2.size(); i++)
      res[i + n] += a2b2[i];
    return res;
  }

  bigint operator*(const bigint &v) const {
    vector<int> a6 = convert_base(this->a, base_digits, 6);
    vector<int> b6 = convert_base(v.a, base_digits, 6);
    vll a(a6.begin(), a6.end());
    vll b(b6.begin(), b6.end());
    while (a.size() < b.size())
      a.push_back(0);
    while (b.size() < a.size())
      b.push_back(0);
    while (a.size() & (a.size() - 1))
      a.push_back(0), b.push_back(0);
    vll c = karatsubaMultiply(a, b);
    bigint res;
    res.sign = sign * v.sign;
    for (int i = 0, carry = 0; i < (int) c.size(); i++) {
      long long cur = c[i] + carry;
      res.a.push_back((int) (cur % 1000000));
      carry = (int) (cur / 1000000);
    }
    res.a = convert_base(res.a, 6, base_digits);
    res.trim();
    return res;
  }
};

int main() {
  ios :: sync_with_stdio(false);
  string A, B; cin >> A >> B;
  bigint a(A);
  bigint b(B);

  cout << a + b << endl;
  cout << a - b << endl;
  cout << a * b << endl;
}
```

• 5.2 __int128

```
istream& operator>>(istream& in, __int128_t &n) {
```

```
  string s;
  if (!(in >> s)) return in;
  n = 0;
  int i = 0, sign = 1;
  if (s[0] == '-') {
    sign = -1;
    i = 1;
  } else if (s[0] == '+') {
    i = 1;
  }
  for (; i < s.length(); ++i) {
    if (s[i] >= '0' && s[i] <= '9') {
      n = n * 10 + (s[i] - '0');
    }
  }
  n *= sign;
  return in;
}

ostream& operator<<(ostream& out, __int128_t n) {
  if (n == 0) return out << 0;
  if (n < 0) {
    out << '-';
    n = -n;
  }
  string s = "";
  while (n > 0) {
    s += (char)('0' + (n % 10));
    n /= 10;
  }
  for (int i = (int)s.length() - 1; i >= 0; --i) {
    out << s[i];
  }
  return out;
}
```

6 Flow

• 6.1 Dinic

```
struct FlowEdge {
  int v, u;
  long long cap, flow = 0;
  FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(
    cap) {}
};

struct Dinic {
  const long long flow_inf = 1e18;
  vector<FlowEdge> edges;
  vector<vector<int>> adj;
  int n, m = 0;
  int s, t;
  vector<int> level, ptr;
  queue<int> q;

  Dinic(int n, int s, int t) : n(n), s(s), t(t) {
    adj.resize(n);
    level.resize(n);
    ptr.resize(n);
  }

  void add_edge(int v, int u, long long cap) {
    edges.emplace_back(v, u, cap);
    edges.emplace_back(u, v, 0);
    adj[v].push_back(m);
    adj[u].push_back(m + 1);
    m += 2;
  }

  bool bfs() {
    while (!q.empty()) {
      int v = q.front();
      q.pop();
      for (int id : adj[v]) {
        if (edges[id].cap == edges[id].flow)
          continue;
        if (level[edges[id].u] != -1)
          continue;
        level[edges[id].u] = level[v] + 1;
        q.push(edges[id].u);
      }
    }
    return level[t] != -1;
  }

  long long dfs(int v, long long pushed) {
    if (pushed == 0)
      return 0;
    if (v == t)
      return pushed;
    for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++)
     {
      int id = adj[v][cid];
      int u = edges[id].u;
      if (level[v] + 1 != level[u])
        continue;
      long long tr = dfs(u, min(pushed, edges[id].cap -
    edges[id].flow));
      if (tr == 0)
        continue;
      edges[id].flow += tr;
      edges[id ^ 1].flow -= tr;
      return tr;
    }
    return 0;
  }

  long long flow() {
    long long f = 0;
    while (true) {
      fill(level.begin(), level.end(), -1);
      level[s] = 0;
      q.push(s);
      if (!bfs())
        break;
      fill(ptr.begin(), ptr.end(), 0);
      while (long long pushed = dfs(s, flow_inf)) {
        f += pushed;
      }
    }
    return f;
  }
};
```

7 Geometry

• 7.1 Convex Hull

```
struct Point {
  ll x, y;

  Point(ll x = 0, ll y = 0) : x(x), y(y) {}
};

struct Vector {
  ll x, y;

  Vector(Point A, Point B) : x(B.x - A.x), y(B.y - A.y) {}

  ll cross(const Vector &ot) const {
    return x * ot.y - ot.x * y;
  }
};

bool cw(const Point &A, const Point &B, const Point &C) {
  return Vector(A, B).cross(Vector(A, C)) < 0;
}

int n;
vector <Point> a;

vector <Point> Convex_hull() {
  vector <Point> lower, upper;
  sort(a.begin(), a.end(), [] (const Point &A, const Point &
    B) {
    if (A.x == B.x) return A.y < B.y;
    return A.x < B.x;
  });

  lower.pb(a[0]);
  for (int i = 1; i < n; i++) {
    while (Sz(lower) >= 2 and cw(lower[Sz(lower) - 2], lower
    .back(), a[i])) lower.pop_back();
    lower.pb(a[i]);
  }

  upper.pb(a[n - 1]);
  for (int i = n - 2; i >= 0; i--) {
    while (Sz(upper) >= 2 and cw(upper[Sz(upper) - 2], upper
    .back(), a[i])) upper.pop_back();
    upper.pb(a[i]);
  }

  if (Sz(lower) + Sz(upper) == 2) return lower;
  lower.pop_back();
  upper.pop_back();
  vector <Point> hull = lower;
  hull.insert(hull.end(), upper.begin(), upper.end());
  return hull;
}
```

8 Fast input - output

• 8.1 Fast Input

```
static struct FastInput {
  static constexpr int BUF_SIZE = 1 << 20;
  char buf[BUF_SIZE];
  size_t chars_read = 0;
  size_t buf_pos = 0;
  FILE *in = stdin;
  char cur = 0;

    inline char get_char() {
    if (buf_pos >= chars_read) {
    chars_read = fread(buf, 1, BUF_SIZE, in);
    buf_pos = 0;
    buf[0] = (chars_read == 0 ? -1 : buf[0]);
    }
    return cur = buf[buf_pos++];
  }

  inline void tie(int) {}

  inline explicit operator bool() {
    return cur != -1;
  }

  inline static bool is_blank(char c) {
    return c <= ' ';
  }

  inline bool skip_blanks() {
    while (is_blank(cur) && cur != -1) {
    get_char();
    }
    return cur != -1;
```

```
  }

  inline FastInput& operator>>(char& c) {
    skip_blanks();
    c = cur;
    return *this;
  }

  inline FastInput& operator>>(string& s) {
    if (skip_blanks()) {
    s.clear();
    do {
      s += cur;
    } while (!is_blank(get_char()));
    }
    return *this;
  }

  template <typename T>
  inline FastInput& read_integer(T& n) {
    // unsafe, doesn't check that characters are actually
    digits
    n = 0;
    if (skip_blanks()) {
    int sign = +1;
    if (cur == '-') {
      sign = -1;
      get_char();
    }
    do {
      n += n + (n << 3) + cur - '0';
    } while (!is_blank(get_char()));
    n *= sign;
    }
    return *this;
  }

  template <typename T>
  inline typename enable_if<is_integral<T>::value, FastInput
    &>::type operator>>(T& n) {
    return read_integer(n);
  }

  #if !defined(_WIN32) | defined(_WIN64)
  inline FastInput& operator>>(__int128& n) {
    return read_integer(n);
  }
  #endif

  template <typename T>
  inline typename enable_if<is_floating_point<T>::value,
    FastInput&>::type operator>>(T& n) {
    // not sure if really fast, for compatibility only
    n = 0;
    if (skip_blanks()) {
    string s;
    (*this) >> s;
    sscanf(s.c_str(), "%lf", &n);
    }
    return *this;
  }
} fast_input;

#define cin fast_input
```

• 8.2 Fast Output

```
static struct FastOutput {
  static constexpr int BUF_SIZE = 1 << 20;
  char buf[BUF_SIZE];
  size_t buf_pos = 0;
  static constexpr int TMP_SIZE = 1 << 20;
  char tmp[TMP_SIZE];
  FILE *out = stdout;

  inline void put_char(char c) {
    buf[buf_pos++] = c;
    if (buf_pos == BUF_SIZE) {
    fwrite(buf, 1, buf_pos, out);
    buf_pos = 0;
    }
  }

  ~FastOutput() {
    fwrite(buf, 1, buf_pos, out);
  }

  inline FastOutput& operator<<(char c) {
    put_char(c);
    return *this;
  }

  inline FastOutput& operator<<(const char* s) {
    while (*s) {
    put_char(*s++);
    }
    return *this;
  }

  inline FastOutput& operator<<(const string& s) {
    for (int i = 0; i < (int) s.size(); i++) {
    put_char(s[i]);
    }
    return *this;
  }

  template <typename T>
  inline char* integer_to_string(T n) {
    char* p = tmp + TMP_SIZE - 1;
    if (n == 0) {
    *--p = '0';
    } else {
    bool is_negative = false;
    if (n < 0) {
      is_negative = true;
      n = -n;
    }
    while (n > 0) {
      *--p = (char) ('0' + n % 10);
      n /= 10;
    }
    if (is_negative) {
      *--p = '-';
    }
    }
    return p;
  }

  template <typename T>
  inline typename enable_if<is_integral<T>::value, char*>::
    type stringify(T n) {
    return integer_to_string(n);
  }

  #if !defined(_WIN32) || defined(_WIN64)
  inline char* stringify(__int128 n) {
    return integer_to_string(n);
  }
  #endif

  template <typename T>
  inline typename enable_if<is_floating_point<T>::value,
    char*>::type stringify(T n) {
    sprintf(tmp, "%.17f", n);
    return tmp;
  }

  template <typename T>
  inline FastOutput& operator<<(const T& n) {
    auto p = stringify(n);
    for (; *p != 0; p++) {
    put_char(*p);
    }
    return *this;
  }
} fast_output;

#define cout fast_output
```